

Học Sâu (Deep Learning)

Chương 13: Dự báo Chuỗi thời gian (Timeseries Forecasting)

Đại học Đông Á

August 14, 2026

Nội dung chương

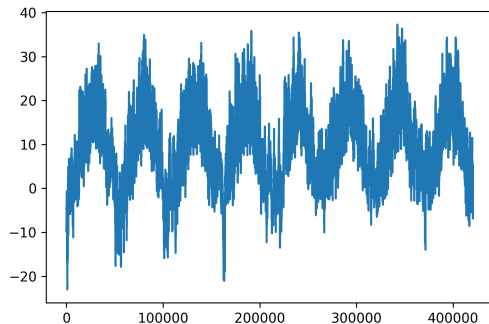
1. Giới thiệu Chuỗi thời gian và Bài toán Dự báo
2. Sự thất bại của các Mạng truyền thống
3. Mạng Nơ-ron Truyền hồi (Recurrent Neural Networks - RNN)
4. Kiến trúc LSTM - Long Short-Term Memory
5. Các Kỹ thuật Nâng cao cho RNN
6. Tổng kết

Chuỗi thời gian (Timeseries) là gì?

- Chuỗi thời gian là tập hợp các điểm dữ liệu được ghi nhận theo những khoảng thời gian đều đặn.
- Yếu tố "**Thứ tự**" (Temporal Order) mang tính quyết định. Quá khứ ảnh hưởng đến tương lai.
- Bài toán phổ biến nhất: **Dự báo (Forecasting)** - Dự đoán điều gì sẽ xảy ra tiếp theo.
- Ứng dụng: Dự báo tiêu thụ điện năng, dự báo doanh thu, dự báo thời tiết, hoặc giao dịch chứng khoán.

Khảo sát Dữ liệu Thời tiết Jena (Dài hạn)

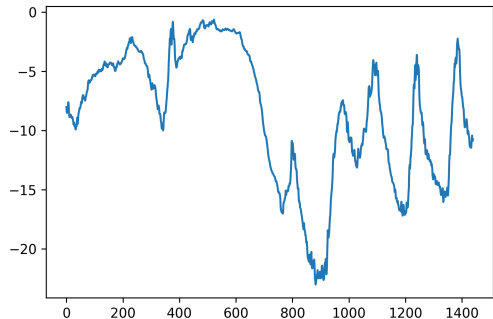
- Bộ dữ liệu từ Viện Max Planck, Đức. Ghi nhận 14 thông số thời tiết mỗi 10 phút.
- Trực quan hóa Biểu đồ Nhiệt độ trong khoảng thời gian nhiều năm.
- **Đặc điểm:** Có tính chu kỳ (Seasonality) rất rõ rệt theo các năm (Mùa đông lạnh, mùa hè nóng).



Hình 13.1: Nhiệt độ thay đổi qua nhiều năm

Khảo sát Dữ liệu Thời tiết Jena (Ngắn hạn)

- Trực quan hóa chi tiết trong vòng 10 ngày (1440 điểm dữ liệu đầu tiên).
- Vẫn quan sát được tính chu kỳ theo ngày (Ngày ấm, đêm lạnh).
- Tuy nhiên, dữ liệu rất nhiễu và biến động mạnh do các yếu tố ngẫu nhiên.
- **Mục tiêu:** Dựa vào dữ liệu quá khứ, dự báo nhiệt độ của 24 giờ tiếp theo.



Hình 13.2: Nhiệt độ biến động trong vài ngày

Tải và Đọc Dữ liệu Jena Weather

Dữ liệu thời tiết Jena được lưu dưới định dạng csv. Chúng ta dùng Python để đọc dòng dữ liệu:

```
1 import os
2
3 # Define the file path
4 fname = os.path.join("jena_climate_2009_2016.csv")
5
6 # Read the file
7 with open(fname) as f:
8     data = f.read()
9
10 # Split by newlines
11 lines = data.split("\n")
12 header = lines[0].split(",")
13 lines = lines[1:]
14
15 print("Header features:", header)
16 print("Total timesteps:", len(lines)) # 420,551 lines
```

Chuyển đổi dữ liệu sang Numpy Array

Chuyển đổi file CSV thành 2 mảng Numpy: mảng temperature dùng làm Target (mục tiêu dự báo) và raw_data làm Input Features.

```
1 import numpy as np
2
3 # Arrays for features and targets
4 temperature = np.zeros((len(lines),))
5 raw_data = np.zeros((len(lines), len(header) - 1))
6
7 for i, line in enumerate(lines):
8     # Skip the Date Time column (index 0)
9     values = [float(x) for x in line.split(",")[1:]]
10
11     # Column 1 is temperature (T (degC))
12     temperature[i] = values[1]
13
14     # Store all columns in raw_data (including temperature)
15     raw_data[i, :] = values[:]
```

Phân chia tập dữ liệu (Train/Val/Test)

Khi làm việc với chuỗi thời gian, ta **không được xáo trộn ngẫu nhiên (shuffle)** các mẫu. Thay vào đó, tập kiểm tra phải đến **sau** tập huấn luyện trên trục thời gian.

```
1 # First 50% for training
2 num_train_samples = int(0.5 * len(raw_data))
3
4 # Next 25% for validation
5 num_val_samples = int(0.25 * len(raw_data))
6
7 # Remaining 25% for testing
8 num_test_samples = len(raw_data) - num_train_samples - num_val_samples
9
10 print("num_train_samples:", num_train_samples)
11 print("num_val_samples:", num_val_samples)
12 print("num_test_samples:", num_test_samples)
```


Chuẩn hóa Dữ liệu (Normalization)

Các đại lượng đo lường có đơn vị và biên độ khác nhau (áp suất khoảng 1000 mbar, độ ẩm phần trăm từ 0 – 100). Chuẩn hóa (Z-score) giúp mô hình hội tụ nhanh hơn.

```
1 # Calculate mean and standard deviation ONLY on the training split
2 # to prevent data leakage from the future
3 mean = raw_data[:num_train_samples].mean(axis=0)
4 raw_data -= mean
5
6 std = raw_data[:num_train_samples].std(axis=0)
7 raw_data /= std
```

- **Lưu ý quan trọng:** Tuyệt đối không tính mean hay std trên toàn bộ raw_data. Nó vi phạm nguyên tắc "Mô hình không được biết thông tin của tương lai".

Định nghĩa tham số Cửa sổ Thời gian (Time Window)

Mục tiêu: Dùng dữ liệu 5 ngày liên tục (120 giờ) để dự báo thời tiết của 24 giờ kế tiếp.

```
1 # Observations are sampled every 10 minutes (6 times an hour)
2 sampling_rate = 6
3
4 # We use the past 5 days (120 hours) as our input sequence
5 sequence_length = 120
6
7 # We want to predict the temperature 24 hours into the future
8 delay = sampling_rate * (sequence_length + 24 - 1)
9
10 # Batch size for neural networks
11 batch_size = 256
```

Khởi tạo Timeseries Generator của Keras

Hàm `timeseries_dataset_from_array` giúp tạo tự động các chuỗi nối tiếp từ dữ liệu thô mà không tốn quá nhiều RAM.

```
1 import keras
2
3 train_dataset = keras.utils.timeseries_dataset_from_array(
4     raw_data[:-delay],
5     targets=temperature[delay:],
6     sampling_rate=sampling_rate,
7     sequence_length=sequence_length,
8     shuffle=True, # Shuffle BATCHES, not the timeline itself
9     batch_size=batch_size,
10    start_index=0,
11    end_index=num_train_samples,
12 )
```

Tương tự, ta tạo `val_dataset` và `test_dataset` bằng cách thay đổi `start_index` và `end_index`.

Một Baseline Suy luận Thông thường (Commonsense Baseline)

Trực giác con người: Thời tiết ngày mai sẽ khá giống thời tiết hôm nay. Vậy nếu dự báo "Nhiệt độ 24h tới = Nhiệt độ hiện tại" thì sai số là bao nhiêu?

```
1 def evaluate_naive_method(dataset):
2     total_abs_err = 0.0
3     samples_seen = 0
4     for samples, targets in dataset:
5         # Predict the last temperature in the sequence
6         # Temperature is column 1 (index 1)
7         preds = samples[:, -1, 1] * std[1] + mean[1]
8
9         # Calculate Absolute Error
10        total_abs_err += np.sum(np.abs(preds - targets))
11        samples_seen += samples.shape[0]
12    return total_abs_err / samples_seen
13
14 print(f"Validation MAE: {evaluate_naive_method(val_dataset):.2f}")
```

Sai số trung bình tuyệt đối (MAE) của phương pháp ngây ngô này là **2.44 độ C**. Bất kì AI nào muốn hoạt động hiệu quả cũng phải có Loss thấp hơn con số này.

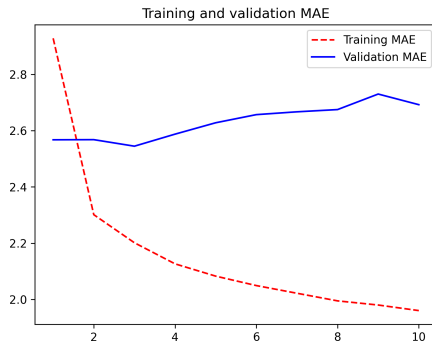
Sự thất bại của mạng Densely-connected Model

Thử nghiệm Feed-forward Network truyền thống. Chúng ta trải phẳng (Flatten) mảng thời gian thành Vector 1D.

```
1 from keras import layers
2
3 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
4 # Flatten the sequence: destroys the concept of "time"
5 x = layers.Flatten()(inputs)
6 x = layers.Dense(16, activation="relu")(x)
7 outputs = layers.Dense(1)(x) # Output is continuous prediction
8
9 model = keras.Model(inputs, outputs)
10 model.compile(optimizer="adam", loss="mse", metrics=["mae"])
11
12 history = model.fit(train_dataset, epochs=10,
13                     validation_data=val_dataset)
```

Biểu đồ Loss của Densely-connected Model

- Trải phẳng (Flatten) chuỗi thời gian và đưa qua các lớp Dense.
- **Kết quả:** Loss trên tập Validation (Đường liền) bị bão hòa nhanh chóng, Overfit. Validation MAE khoảng 2.6 độ C, tệ hơn cả Baseline (2.44).
- **Nguyên nhân:** Mạng Dense loại bỏ hoàn toàn khái niệm "Thời gian". Nó xem mọi điểm dữ liệu trong chuỗi là độc lập (IID).



Hình 13.3: Training Loss của Dense Model

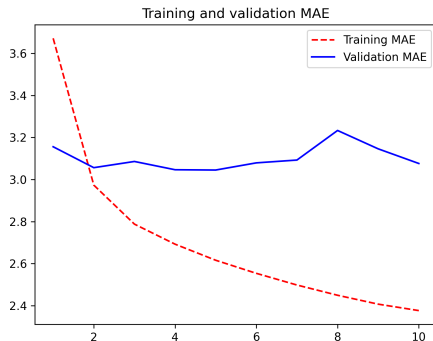
Thử nghiệm với Mạng 1D Convolution

Thay đổi kiến trúc bằng mạng Tích chập 1 chiều (1D ConvNet) với hi vọng trích xuất đặc trưng lặp lại cục bộ.

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2 # Swipe a 1D window across sequence
3 x = layers.Conv1D(8, 24, activation="relu")(inputs)
4 x = layers.MaxPooling1D(2)(x)
5 x = layers.Conv1D(8, 12, activation="relu")(x)
6 x = layers.MaxPooling1D(2)(x)
7 x = layers.Conv1D(8, 6, activation="relu")(x)
8 x = layers.GlobalAveragePooling1D()(x) # Collapse time dimension
9
10 outputs = layers.Dense(1)(x)
11 model = keras.Model(inputs, outputs)
12 model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Biểu đồ Loss của Mạng 1D Convolution

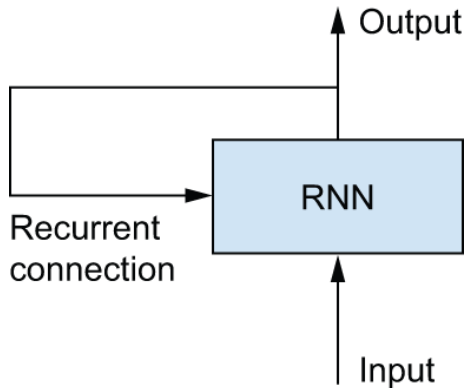
- **Kết quả:** Validation MAE rơi vào khoảng 2.9 độ C. Tệ hơn cả Mạng Dense!
- **Lý do:** Conv1D giả định ranh giới Dịch chuyển Bất biến (Translation Invariance). Nghĩa là một khuôn mẫu ở thứ 2 có ý nghĩa giống hệt thứ 5.
- Điều này sai lầm trong Timeseries vì thời gian **càng gần hiện tại càng mang thông tin quan trọng**. Tính chất Max Pooling cũng vô tình xóa sạch khái niệm thứ tự (Order).



Hình 13.4: Training Loss của 1D Conv

Khái niệm Mạng Nơ-ron Truyền hồi (RNN)

- Trái ngược với mạng Feed-forward, RNN duy trì một **Trạng thái (State)** đóng vai trò như "Bộ nhớ".
- Nó duyệt qua từng bước thời gian t , tính toán Đầu ra (Output) tại t , và cập nhật lại Trạng thái để dùng cho $t + 1$.
- RNN lưu giữ thông tin về những gì nó đã thấy cho đến nay.



Hình 13.5: Vòng lặp trong mạng SimpleRNN

Vòng lặp bên trong mô hình RNN (Giả lập)

Khác với vòng lặp for bình thường, trạng thái của một vòng lặp RNN được lưu vết lại để tính cho bước tiếp theo.

```
1 state_t = 0 # The state at time t
2 for input_t in input_sequence:
3     # Function f combines input and current state using weights
4     # output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
5     output_t = f(input_t, state_t)
6
7     # The previous output becomes the state for the next iteration.
8     state_t = output_t
```

Trạng thái này được khởi tạo bằng 0 lúc bắt đầu một chuỗi.

Lớp SimpleRNN trong Keras

Mặc định RNN chỉ trả về Trạng thái Ẩn cuối cùng (Last State Output) mang theo toàn bộ ký ức. Nếu muốn giữ nguyên các trạng thái trung gian, cấu hình `return_sequences`.

```
1 num_features = 14
2 inputs = keras.Input(shape=(None, num_features))
3
4 # 1. Return ONLY the final output (Shape: (None, 16))
5 outputs = layers.SimpleRNN(16, return_sequences=False)(inputs)
6
7 # 2. Return the FULL sequence (Shape: (None, timesteps, 16))
8 outputs = layers.SimpleRNN(16, return_sequences=True)(inputs)
9
10 # Stacking RNNs requires return_sequences=True
11 x = layers.SimpleRNN(16, return_sequences=True)(inputs)
12 x = layers.SimpleRNN(16, return_sequences=True)(x)
13 outputs = layers.SimpleRNN(16)(x)
```

Vấn đề cốt lõi: Vanishing Gradients (Triệt tiêu Đạo hàm)

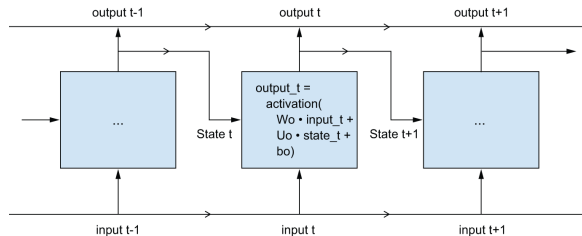
- Mạng **SimpleRNN** hoạt động tốt về mặt lý thuyết, nhưng thất bại trong thực tế.
- Khi độ dài chuỗi tăng lên, Đạo hàm (Gradient) từ tương lai phải nhân liên tục qua hàng trăm ma trận trọng số để quay ngược về quá khứ.
- Nếu các ma trận trọng số nhỏ hơn 1, đạo hàm sẽ tiến về 0 cực nhanh (Triệt tiêu).
- Hậu quả: SimpleRNN bị "mất trí nhớ" ngắn hạn, quên mất các thông tin đầu chuỗi. Mạng dừng học.

Giải pháp LSTM: Bổ sung Trạng thái Tế bào (Cell State)

- Được phát minh năm 1997, LSTM bổ sung một đường truyền dữ liệu song song gọi là **Cell State (C_t)**.
- Tưởng tượng C_t như một băng chuyền chạy dọc qua toàn bộ chuỗi. Thông tin có thể dễ dàng chảy xuyên suốt mà không bị thay đổi quá nhiều (Giải quyết hoàn toàn Vanishing Gradients).
- Các **Cổng (Gates)** sẽ quyết định thêm/bớt thông tin gì vào dải băng chuyền này.

LSTM (Phần 1): Trạng thái mang theo

- Tại thời điểm t , ta có Đầu vào X_t và Trạng thái ẩn từ quá khứ H_{t-1} .
- Thay vì chỉ đưa ra H_t như RNN thông thường, LSTM truyền thêm C_{t-1} vào ô tế bào hiện tại.



Hình 13.6: Giai đoạn khởi nhập của LSTM

LSTM (Phần 2): Forget Gate và Input Gate

Tính toán Tế bào trạng thái mới C_t :

- 1 **Forget Gate (f):** Quên bớt thông tin cũ không còn quan trọng. Hàm Sigmoid xuất giá trị 0 (Xóa) đến 1 (Giữ lại).

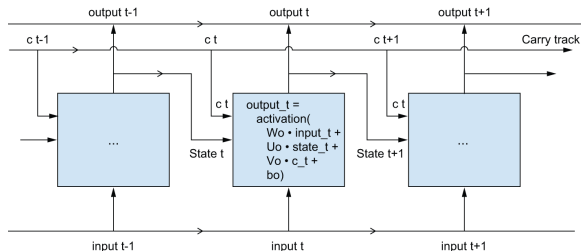
$$f = \sigma(W_f \cdot [X_t, H_{t-1}] + b_f)$$

- 2 **Input Gate (i và k):** Tạo thông tin mới để cập nhật vào băng chuyển.

$$i = \sigma(W_i \cdot [X_t, H_{t-1}] + b_i)$$

$$k = \tanh(W_k \cdot [X_t, H_{t-1}] + b_k)$$

- 3 $C_t = C_{t-1} * f + i * k$



Hình 13.7: Cập nhật Cell State c_t

LSTM (Phần 3): Output Gate

Cuối cùng, tính Trạng thái ẩn H_t (Cũng chính là Output):

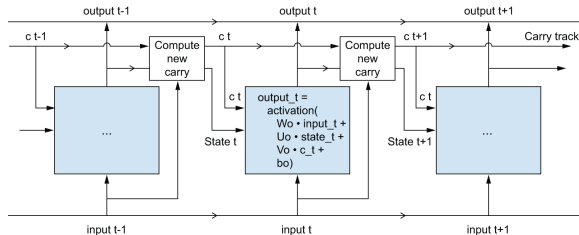
- **Output Gate (o):** Quyết định phần nào của Cell State sẽ được xuất ra.

$$o = \sigma(W_o \cdot [X_t, H_{t-1}] + b_o)$$

- Tính toán H_t :

$$H_t = o * \tanh(C_t)$$

- Trạng thái ẩn H_t mang thông tin tinh lọc nhất để chuẩn bị bước sang thời điểm $t + 1$.



Hình 13.8: Xuất giá trị h_t

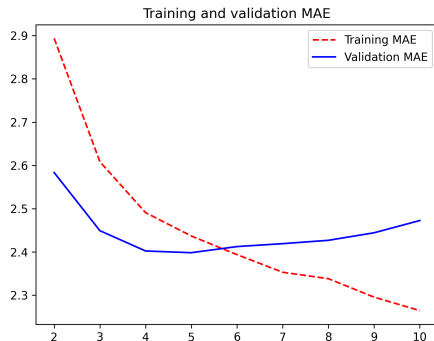
Xây dựng mô hình LSTM với Keras

Mặc dù thuật toán phức tạp, nhưng việc triển khai lại cực kỳ đơn giản với Keras qua class LSTM.

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2
3 # The LSTM layer handles all the internal memory cell logic
4 x = layers.LSTM(16)(inputs)
5 outputs = layers.Dense(1)(x)
6
7 model = keras.Model(inputs, outputs)
8 model.compile(optimizer="adam", loss="mse", metrics=["mae"])
9
10 history = model.fit(train_dataset, epochs=10,
11                     validation_data=val_dataset)
```

Hiệu suất Đột phá của LSTM

- **Kết quả:** Validation MAE giảm còn 2.39 độ C, chính thức **vượt qua độ chính xác của Baseline** (2.44 độ C).
- Tuy nhiên, do số lượng tham số lớn, mô hình bắt đầu bị hiện tượng Overfit từ Epoch 5 (Validation Loss cong dần lên).
- Cần các kỹ thuật nâng cao tiếp theo.



Hình 13.9: Biểu đồ Loss của LSTM

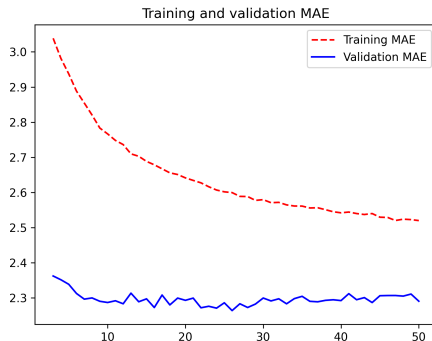
Kỹ thuật 1: Recurrent Dropout (Chống Overfitting)

Dropout truyền thống có thể phá vỡ tính liên tục của thời gian. Keras cung cấp tham số `recurrent_dropout` để tắt ngẫu nhiên nơ-ron "bên trong" tế bào LSTM thay vì chỉ tắt đầu vào.

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2
3 # Apply recurrent_dropout to the hidden state connections
4 x = layers.LSTM(32, recurrent_dropout=0.25)(inputs)
5
6 # Standard dropout for the Dense head
7 x = layers.Dropout(0.5)(x)
8
9 outputs = layers.Dense(1)(x)
10 model = keras.Model(inputs, outputs)
11 model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Biểu đồ sức mạnh của Recurrent Dropout

- **Kết quả:** Mô hình duy trì độ ổn định tuyệt vời. Đường Validation và Training chạy sát nhau và không phân kì.
- Validation Loss giảm sâu hơn nữa xuống 2.27 độ C (Cải thiện 7% so với Baseline ban đầu).
- Quá trình huấn luyện không còn bị Overfit sớm, có thể tăng Epochs thoải mái.



Hình 13.10: Sức mạnh của Recurrent Dropout

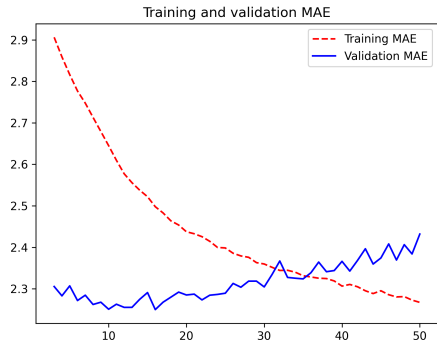
Kỹ thuật 2: Xếp chồng RNN (Stacked GRU)

Khi mô hình không còn Overfit, ta có thể an tâm tăng sức mạnh cấu trúc bằng cách Xếp chồng các lớp RNN. Sử dụng lớp GRU giúp tốc độ tính toán nhanh hơn.

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2
3 # Bottom layer MUST return sequence for the next layer
4 x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
5
6 # Top layer
7 x = layers.GRU(32, recurrent_dropout=0.5)(x)
8 x = layers.Dropout(0.5)(x)
9
10 outputs = layers.Dense(1)(x)
11 model = keras.Model(inputs, outputs)
```

Biểu đồ Xếp chồng GRU + Dropout

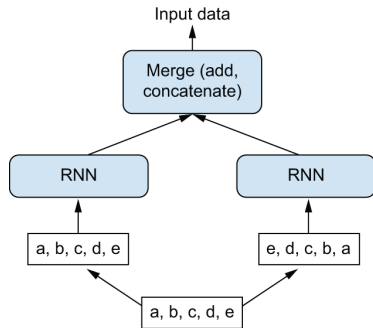
- **Kết quả:** Mô hình tiếp tục hội tụ và cho kết quả ổn định.
- Tăng dung lượng mạng đôi khi gặp phải "Lợi tức giảm dần" (Diminishing Returns), tức là mạng lớn hơn nhưng hiệu suất tăng không đáng kể.
- GRU (Gated Recurrent Unit) là phiên bản tinh giản của LSTM nhưng đem lại hiệu quả tương đương.



Hình 13.11: Xếp chồng GRU + Dropout

Kỹ thuật 3: Mạng truyền hồi Hai chiều (Bidirectional RNN)

- RNN thông thường chỉ đọc dữ liệu theo chiều thời gian (Từ quá khứ tới hiện tại).
- Bidirectional RNN dùng 2 lớp RNN độc lập: 1 đọc xuôi, 1 đọc ngược (từ tương lai về quá khứ), sau đó gộp kết quả lại.
- Vô cùng hữu ích trong xử lý ngôn ngữ tự nhiên (NLP) khi ý nghĩa của một từ phụ thuộc vào những từ phía sau nó.



Hình 13.12: Kiến trúc Bidirectional RNN

Xây dựng Bidirectional LSTM

Lớp Bidirectional của Keras đóng vai trò như một wrapper, nhân đôi LSTM bên trong nó và đảo ngược chuỗi thời gian cho nhân bản thứ hai.

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2
3 # Wrap the LSTM layer inside a Bidirectional wrapper
4 x = layers.Bidirectional(layers.LSTM(16))(inputs)
5
6 outputs = layers.Dense(1)(x)
7 model = keras.Model(inputs, outputs)
8
9 model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Tuy nhiên, Bidirectional hoạt động rất kém trên tập dữ liệu Thời tiết, bởi vì yếu tố quá khứ đóng vai trò quan trọng, trong khi chiều đọc ngược (tương lai về quá khứ) chứa dữ liệu gây nhiễu loạn luồng Logic của LSTM.

Tổng kết Chương 13

- **Timeseries (Chuỗi thời gian)** khác biệt hoàn toàn với dữ liệu thông thường vì thứ tự cấu thành nên ý nghĩa. Dense và Conv1D không phù hợp.
- **Mạng truyền hồi (RNN)** duy trì "Trạng thái" bộ nhớ, nhưng vướng phải lỗi Triệt tiêu đạo hàm (Vanishing Gradients).
- **LSTM / GRU** giải quyết triệt để lỗi trên bằng hệ thống Cổng (Gates) và Trạng thái Tế bào (Cell State C_t).
- Các kỹ thuật nâng cao như **Recurrent Dropout**, **Stacked RNN**, và **Bidirectional RNN** giúp tối ưu hóa hiệu suất tối đa.